Отчёт по лабораторной работе №2

Управление версиями

Дмитраков Михаил Алексеевич

Содержание

Содержание 2
1 Цель работы 3
2 Выполнение лабораторной работы 4
3 Вывод 14
4 Контрольные вопросы 15

1 Цель работы

Целью данной работы является изучение идеологии и применения средств контроля версий и освоение умений работать с git.

2 Выполнение лабораторной работы

Устанавливаем git, git-flow и gh.

```
dmitrakovma@dmitrakovma:~$ git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy
           [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
           [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envva
           <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
   clone     Clone a repository into a new directory
   init      Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
   add       Add file contents to the index
   mv        Move or rename a file, a directory, or a symlink
   restore   Restore working tree files
   rm        Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
   bisect    Use binary search to find the commit that introduced a bug
   diff      Show changes between commits, commit and working tree, etc
   grep      Print lines matching a pattern
   log       Show commit logs
   show      Show various types of objects
   status    Show the working tree status

grow, mark and tweak your common history
   backfill  Download missing objects in a partial clone
   branch    List, create, or delete branches
   commit    Record changes to the repository
```

2.1: Загрузка пакетов

Зададим имя и email владельца репозитория, кодировку и прочие параметры.

```
dmitrakovma@dmitrakovma:~$
dmitrakovma@dmitrakovma:~$ git config --global user.name "dmitrakovma"
dmitrakovma@dmitrakovma:~$ git config --global user.email "1132250457@rudn.r
dmitrakovma@dmitrakovma:~$
dmitrakovma@dmitrakovma:~$ git config --global core.quotepath false
dmitrakovma@dmitrakovma:~$ git config --global init.defaultBranch master
dmitrakovma@dmitrakovma:~$ git config --global core.autocrlf input
dmitrakovma@dmitrakovma:~$ git config --global core.safecrlf warn
dmitrakovma@dmitrakovma:~$
```

Рисунок 2.2: Параметры репозитория

Создаем SSH ключи

```
dmitrakovma@dmitrakovma:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/dmitrakovma/.ssh/id_rsa):
Created directory '/home/dmitrakovma/.ssh'.
Enter passphrase for "/home/dmitrakovma/.ssh/id_rsa" (empty for no passphr
Enter same passphrase again:
Your identification has been saved in /home/dmitrakovma/.ssh/id_rsa
Your public key has been saved in /home/dmitrakovma/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:ioPT2Yav9sH54jm@rqFChGoKt4rcSXT0Jb03JIsueG8 dmitrakovma@dmitrakovma
The key's randomart image is:
+---[RSA 4096]----+
|                 |
|                 |
|.    . 0 .       |
|..  . . =        |
|o  . . +So       |
|oooo.*oo= .      |
|=.+oBoBo .       |
|=.+=+E++         |
|o+o+*0*0.        |
+----[SHA256]-----+
dmitrakovma@dmitrakovma:~$
```

Рисунок 2.3: rsa-4096

```
dmitrakovma@dmitrakovma:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/dmitrakovma/.ssh/id_ed25519):
Enter passphrase for "/home/dmitrakovma/..ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/dmitrakovma/.ssh/id_ed25519
Your public key has been saved in /home/dmitrakovma/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:x3X6+uoA2GguXFyxCrHsauODyfOVF9q8pOwClSy0kDQ dmitrakovma@dmitrakovma
The key's randomart image is:
+--[ED25519 256]--+
|.E  .   .        |
|o... o   o       |
|o o =   o   . .  |
| o = o * . . o   |
|  o . B.S o .    |
| . o += .o   .   |
|.o= o+.=  .   .  |
|o=.oo.+ .  . .   |
|  +oo+ .   .+o.  |
+----[SHA256]-----+
```

Рисунок 2.4: ed25519

Создаем GPG ключ

```
Real name: dmitrakovma687
Email address: 1132250457@rudn.ru
Comment:
You selected this USER-ID:
    "dmitrakovma687 <1132250457@rudn.ru>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? O
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/dmitrakovmma/.gnupg/trustdb.gpg: trustdb created
gpg: directory '/home/dmitrakovma/.gnupg/openpgp-revvocs.d' created
gpg: revocation certificate stored as '/home/dmitrakovmma/.gnupg/openpgp-revocs.d/90CA5
2C642BAE3A2F4626D2D2D203A6469C94834.rev'
public and secret key created and signed.

pub   rsa4096 2026-02-23 [SC]
      90CA52C642BAE3A2F4626D2D2D203A6469C94834
uid                      dmitrakovma687 <1132250457@rudn.ru>
sub   rsa4096 2026-02-23 [E]
```

Рисунок 2.5: GPG ключ

Добавляем GPG ключ в аккаунт

```
dmitrakovma@dmitrakovma:~$
dmitrakovma@dmitrakovma:~$ gpg --list-secret-keys --keyid-format LONG
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:  1  signed:  0  trust: 0-, 0q, 0n, 0m, 0f, 1u
[keyboxd]
---------
sec   rsa4096/2D203A6469C94834 2026-02-23 [SC]
      98CA52C642BAE3A2F4626D2D2D203A6469C94834
uid                 [ultimate] dmitrakovma687 <1132250457@rudn.ru>
ssb   rsa4096/DF7AF054C17A838F 2026-02-23 [E]

dmitrakovma@dmitrakovma:~$ gpg --armor --export 2D203A6469C94834 | xclip -sel clip
dmitrakovma@dmitrakovma:~$
```

Рисунок 2.6: GPG ключ

Настройка автоматических подписей коммитов git

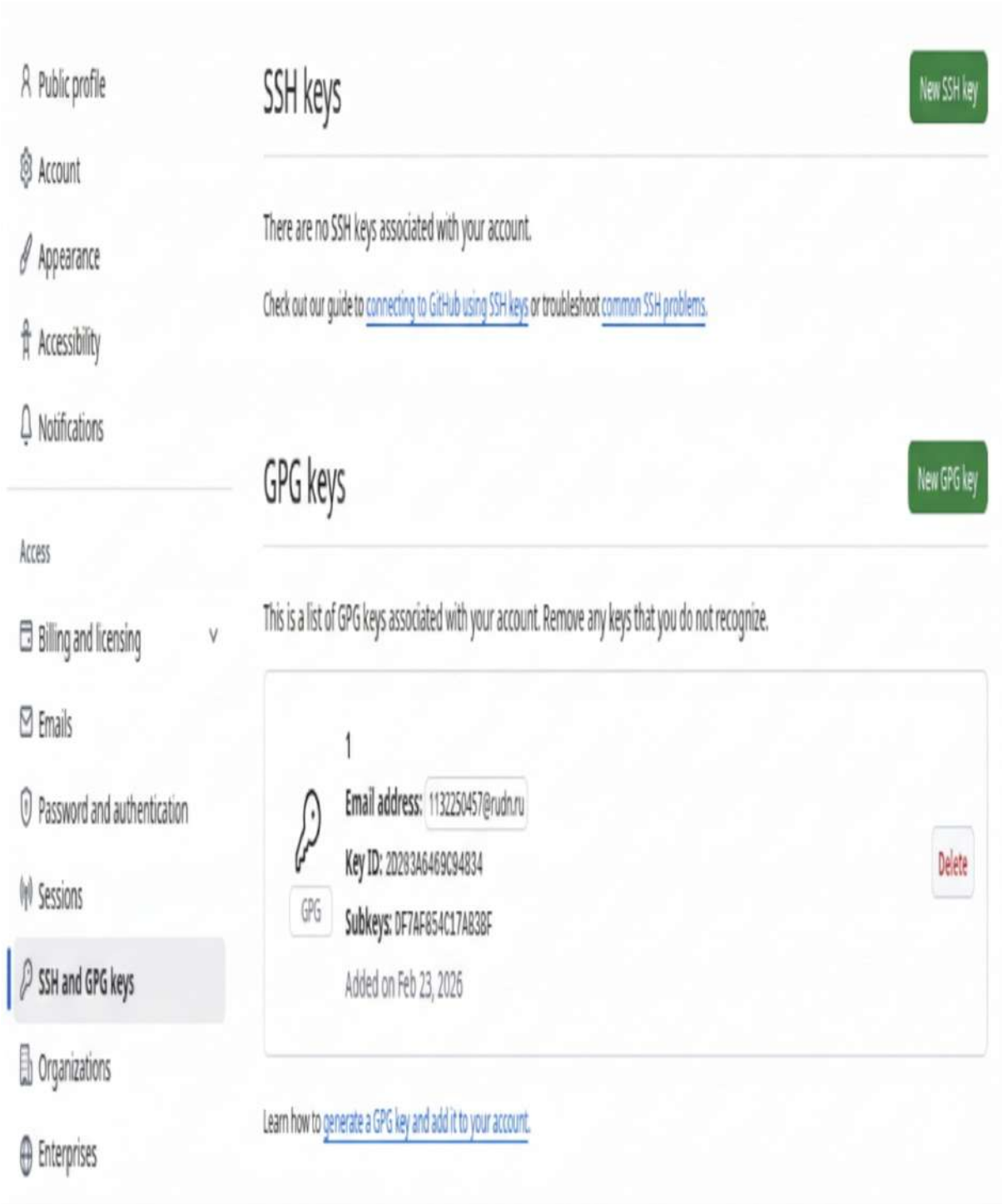

Рисунок 2.7: Параметры репозитория

Настройка gh

```
dmitrakovnma@dmitrakovma:~$ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? SSH
? Upload your SSH public key to your GitHub account? /home/dmitrakovnma/.ssh/id_rsa.pub
? Title for your SSH key: GitHub CLI
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: D9BD-B8BC
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol ssh
✓ Configured git protocol
✓ Uploaded the SSH key to your GitHub account: /home/dmitrakovnma/.ssh/id_rsa.pub
✓ Logged in as dmitrakovnma687
dmitrakovma@dmitrakovma:~$
```

Рисунок 2.8: Связь репозитория с аккаунтом

Загрузка шаблона репозитория и синхронизация

```
dmitrakovma@dmitrakovma:~/work/study/2025-2026/Операционные системы$ cd ~/work/study/
2025-2026/"Операционные системы/2026-1--study--os-intro
dmitrakovma@dmitrakovma:~/work/study/2025-2026/Операционные системы/2026-1--study--os
-intro$ echo os-intro > COURSE
dmitrakovma@dmitrakovma:~/work/study/2025-2026/Операционные системы/2026-1--study--os
-intro$ make prepare
Synchronizing submodule url for 'template/report'
Synchronizing submodule url for 'template/presentation'
Synchronizing submodule url for 'template/presentation'
Synchronizing submodule url for 'template/report'
dmitrakovma@dmitrakovma:~/work/study/2025-2026/Операционные системы/2026-1--study--os
-intro$
```

Рисунок 2.9: Загрузка шаблона

Подготовка репозитория и коммит изменений

```
o/minted-quarto.lua
 create mode 100644 project-personal/stage06/report/_quarto.yml
 create mode 100644 project-personal/stage06/report/_resources/csl/gost-r-7-0-5-
meric.csl
 create mode 100644 project-personal/stage06/report/_resources/tex/preamble.tex
 create mode 100644 project-personal/stage06/report/bib/cite.bib
 create mode 100644 project-personal/stage06/report/image/solvay.jpg
 create mode 100644 project-personal/stage06/report/os-intro-project-personal-st
eport.qmd
dmitrakovma@dmitrakovma:~/work/study/2025-2026/Операционные системы/2026-1--st
-intro$ git push
Enumerating objects: 109, done.
Counting objects: 100% (109/109), done.
Delta compression using up to 8 threads
Compressing objects: 100% (90/90), done.
Writing objects: 100% (106/106), 705.58 KiB | 4.67 MiB/s, done.
Total 106 (delta 41), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (41/41), completed with 1 local object.
To github.com:aagibadullin687/2026-1--study--os-intro.git
   ece4e23..bc5e668  master -> master
dmitrakovma@dmitrakovma:~/work/study/2025-2026/Операционные системы/2026-1--st
-intro$
```

Рисунок 2.10: Первый коммит

3 Вывод

Мы приобрели практические навыки работы с сервисом github.

4 Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.

- хранилище - пространство на накопителе где расположен репозиторий
- commit - сохранение состояния хранилища
- история - список изменений хранилища (коммитов)
- рабочая копия - локальная копия сетевого репозитория, в которой работает программист. Текущее состояние файлов проекта, основанное на версии, загруженной из хранилища (обычно на последней)

3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.

Централизованные системы контроля версий представляют собой приложения типа клиент-сервер, когда репозиторий проекта существует в единственном экземпляре и хранится на сервере. Доступ к нему осуществлялся через специальное клиентское приложение. В качестве примеров таких программных продуктов можно привести CVS, Subversion.

Распределенные системы контроля версий (Distributed Version Control System, DVCS) позволяют хранить репозиторий (его копию) у каждого разработчика, работающего с данной системой. При этом можно выделить центральный репозиторий (условно), в который будут отправляться изменения из локальных и, с ним же эти локальные репозитории будут синхронизироваться.

4. Опишите действия с VCS при единоличной работе с хранилищем.

Один пользователь работает над проектом и по мере необходимости делает коммиты, сохраняя определенные этапы.

5. Опишите порядок работы с общим хранилищем VCS.

Несколько пользователей работают каждый над своей частью проекта. При этом каждый должен работать в своей ветки. При завершении работы ветка пользователя сливается с основной веткой проекта.

6. Каковы основные задачи, решаемые инструментальным средством git?

- Ведение истории версий проекта: журнал (log), метки (tags), ветвления (branches).
- Работа с изменениями: выявление (diff), слияние (patch, merge).
- Обеспечение совместной работы: получение версий с сервера, загрузка обновлений на сервер.

7. Назовите и дайте краткую характеристику командам git.

- git config - установка параметров
- git status - полный список изменений файлов, ожидающих коммита
- git add . - сделать все измененные файлы готовыми для коммита.
- git commit -m "[descriptive message]" - записать изменения с заданным сообщением.
- git branch - список всех локальных веток в текущей директории.
- git checkout [branch-name] - переключиться на указанную ветку и обновить рабочую директорию.
- git merge [branch] - соединить изменения в текущей ветке с изменениями из заданной.
- git push - запушить текущую ветку в удаленную ветку.
- git pull - загрузить историю и изменения удаленной ветки и произвести слияние с текущей веткой.

8. Приведите примеры использования при работе с локальным и удалённым репозиториями.

- git remote add [имя] [url] — добавляет удалённый репозиторий с заданным именем;
- git remote remove [имя] — удаляет удалённый репозиторий с заданным именем;
- git remote rename [старое имя] [новое имя] — переименовывает удалённый репозиторий;
- git remote set-url [имя] [url] — присваивает репозиторию с именем новый адрес;
- git remote show [имя] — показывает информацию о репозитории.

9. Что такое и зачем могут быть нужны ветви (branches)?

Ветвление — это возможность работать над разными версиями проекта: вместо одного списка с упорядоченными коммитами история будет расходиться в определённых точках. Каждая ветвь содержит легковесный указатель HEAD на последний коммит, что позволяет без лишних затрат создать много веток. Ветка по умолчанию называется master, но лучше назвать её в соответствии с разрабатываемой в ней функциональностью.

10.Как и зачем можно игнорировать некоторые файлы при commit?

Зачастую нам не нужно, чтобы Git отслеживал все файлы в репозитории, потому что в их число могут входить: файлы конфигурации с приватными данными, временные файлы, скомпилированные бинарные файлы. Для этого используется файл .gitignore, в котором указываются шаблоны файлов и директорий, которые Git должен игнорировать.